

Cheering Shop – Authentication API

For the mobile/frontend team. Covers customer signup and login, vendor registration, and administrator approval.

Base URL: <https://api.cheeringshop.online/api/v1>

Interactive docs: <https://api.cheeringshop.online/docs>

The server also answers on <https://srv1128440.hstgr.cloud> during the migration. **Use the api.cheeringshop.online address** – the other one will be retired once every client has moved.

Everything below is implemented, deployed and covered by tests. The rest of the API (restaurants, cart, orders, vendor) returns `501 NOT_IMPLEMENTED` with the correct response shape, so you can build against those contracts too.

Contents

1. [Conventions](#) – envelope, tokens, errors
 2. [Registration](#) – the signup journey
 3. [Login](#) – password, biometric, 2FA
 4. [Forgot password](#)
 5. [Session management](#) – refresh, logout
 6. [Profile & security](#)
 7. [Two-factor authentication](#)
 8. [Biometric setup](#) – the crypto contract
 9. [Error reference](#)
 10. [Endpoint summary](#)
 11. [Vendor registration](#) – restaurant owners
 12. [Roles](#) – who can create what
-

1. Conventions

Every response has the same shape

Success

```
{ "success": true, "data": { ... }, "meta": { ... } }
```

Failure

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Invalid input provided",
    "details": ["password: String should have at least 8 characters"]
  }
}
```

Branch on `error.code`, never on `error.message` – messages may be reworded, codes will not. `details` is present only for validation failures.

Tokens

Successful authentication returns:

```
{
  "tokens": {
    "access_token": "eyJhbGciOi...",
    "refresh_token": "eyJhbGciOi...",
    "token_type": "Bearer",
    "expires_in": 1800
  },
  "user": { "id": "...", "role": "CUSTOMER", "email": "...", ... }
}
```

Token	Lifetime	Store in
<code>access_token</code>	30 minutes	memory / secure storage
<code>refresh_token</code>	30 days	Keychain (iOS) / EncryptedSharedPreferences (Android)

Send the access token on every authenticated call:

Authorization: Bearer <access_token>

Refresh tokens rotate. Every call to `/auth/refresh` returns a new refresh token and invalidates the old one. You must persist the new one immediately.

⚠ **Reusing an old refresh token revokes the entire session family** and forces a full re-login. This is deliberate anti-theft behaviour – but it means a buggy client that retries a refresh with a stale token will log the user out. Serialise your refresh calls; never fire two at once.

Request tracing

Every response carries `X-Request-ID`. **Log it**. When reporting a bug, send that value – it ties directly to the server logs for that exact request.

2. Registration

User story

As a new customer, I want to sign up with my email, choose a password, give my name and phone, and optionally turn on Face ID, so I can order food.

1. Enter email	→ POST /auth/otp/send	
2. Enter 4-digit code	→ POST /auth/otp/verify	— tokens —
3. Choose password	→ POST /users/me/password	
4. Name + phone	→ PUT /users/me/profile	
5. Enable Face ID	→ POST /auth/biometrics/enable	(opt)
Registration complete		

You are **authenticated from step 2 onwards** – steps 3–5 all require the `Authorization` header.

Step 1 – Send the code

```
POST /api/v1/auth/otp/send
Content-Type: application/json
```

```
{ "email": "customer@example.com" }
```

Customer signup needs nothing else. Restaurant owners add `"role": "VENDOR"` here – see §11. The role is fixed at creation and cannot be changed later.

```
{ "success": true, "data": { "message": "OTP sent" } }
```

A **4-digit** code is emailed. It expires in **5 minutes**.

Situation	Response
Sent	200
Requested again within 60s	429 + <code>Retry-After</code> header

Show a countdown from the `Retry-After` value before re-enabling "Resend code".

The response is 200 whether or not the address already has an account — that is intentional, so nobody can use this endpoint to discover who is registered. Do not branch on it.

Step 2 – Verify the code

```
POST /api/v1/auth/otp/verify
Content-Type: application/json
```

```
{ "email": "customer@example.com", "code": "4821" }
```

```
{
  "success": true,
  "data": {
    "tokens": { "access_token": "...", "refresh_token": "...", "expires_i
    "user": { "id": "...", "role": "CUSTOMER", "email": "customer@example
              "is_email_verified": true, "full_name": null, "phone": null
  }
}
```

Store both tokens now. The user is signed in.

Situation	Response	<code>error.code</code>
Wrong code	400	INVALID_OTP
Expired	400	INVALID_OTP
3 wrong guesses on one code	429	RATE_LIMITED
10 guesses in an hour	429	RATE_LIMITED

On `RATE_LIMITED` , send them back to step 1 for a fresh code.

Shortcut: you may pass `password` and `full_name` in this same call to collapse steps 2–4 into one request. The stepwise flow below is what your described UI does, so use whichever matches your screens.

Step 3 – Choose a password

```
POST /api/v1/users/me/password
Authorization: Bearer <access_token>

{ "new_password": "MyFirstPass1!" }

{ "success": true, "data": { "message": "Password set", "sessions_revoked"
```

`current_password` is **not required** here – the account has no password yet.

Your session survives this call. `sessions_revoked` is 0 and `tokens` is `null` , meaning nothing changed about your session; keep using the tokens from step 2. (Changing an *existing* password behaves differently – see §6.)

Minimum 8 characters. Enforce it client-side too so users see the error before a round trip.

Step 4 – Name and phone

```
PUT /api/v1/users/me/profile
Authorization: Bearer <access_token>

{ "full_name": "Rahim Uddin", "phone": "+8801712345678" }

{
  "success": true,
  "data": { "id": "...", "full_name": "Rahim Uddin", "phone": "+880171234",
    "is_email_verified": true, "is_phone_verified": false }
}
```

`is_phone_verified` is false , and that is correct. Typing a number is not proof of owning it. The number is stored so a rider can call – verifying it is a separate flow, not yet built.

Situation	Response	error.code
Phone already on another account	409	CONFLICT

⚠ **This is a PUT: omitted fields are cleared.** Sending `{"full_name": "Rahim"}` alone will wipe an existing phone number. Always send the complete profile.

Step 5 – Enable biometrics (optional)

See §8 for the key generation contract. Once you have a public key:

```
POST /api/v1/auth/biometrics/enable
Authorization: Bearer <access_token>

{
  "device_id": "unique-per-install-uuid",
  "device_name": "Rahim's iPhone 15",
  "public_key": "<base64 DER SubjectPublicKeyInfo>",
  "algorithm": "ES256"
}
```

Registration is now complete.

3. Login

User story

As a returning customer, I want to sign in with my password or with Face ID.

3a. Email + password

```
POST /api/v1/auth/login

{ "email": "customer@example.com", "password": "MyFirstPass1!" }
```

Two possible success responses. Check for `requires_2fa` before assuming you have tokens:

```
{ "success": true, "data": { "tokens": {...}, "user": {...} } }
```

```
{ "success": true, "data": { "requires_2fa": true, "temp_token": "...", "
```

If `requires_2fa` is present, go to §7.

Situation	Response	error.code
Wrong password or no such account	401	INVALID_CREDENTIALS
10 failures for one email in 15 min	429	RATE_LIMITED

Unknown-account and wrong-password return **byte-identical** responses, on purpose. Never show "no account with that email" — you cannot tell, and neither should an attacker.

3b. Biometric

Two calls: fetch a challenge, sign it, send the signature.

```
POST /api/v1/auth/biometrics/challenge
```

```
{ "device_id": "unique-per-install-uuid" }
```

```
{ "success": true, "data": { "challenge": "hK3n...random...", "expires_in": 120 }
```

Prompt Face ID / fingerprint, sign the **raw UTF-8 challenge string** with the device private key, then:

```
POST /api/v1/auth/biometrics/login
```

```
{ "device_id": "unique-per-install-uuid", "signature": "<base64 signature"
```

Returns tokens, or a `requires_2fa` challenge if 2FA is enabled.

Situation	Response	error.code
Challenge expired (>2 min) or reused	401	INVALID_CREDENTIALS
Bad signature / unknown device	401	INVALID_CREDENTIALS

Situation	Response	<code>error.code</code>
5 consecutive failures	401	FORBIDDEN – device locked, must re-enrol via password login

Challenges are single-use. Fetch a fresh one for every attempt; never cache. A captured signature cannot be replayed.

4. Forgot password

User story

As a customer who forgot their password, I want to reset it by email.

1. Enter email → POST /auth/password/forgot
2. Enter code ↴
3. Enter new password ↴ → POST /auth/password/reset
4. Sign in again → POST /auth/login

Step 1 – Request a reset code

```
POST /api/v1/auth/password/forgot
```

```
{ "email": "customer@example.com" }
```

```
{ "success": true, "data": { "message": "If an account exists, a reset co
```

Always 200 , always the same message – even for an address with no account. Show that message verbatim; do not attempt to detect whether the account exists.

Step 2 – Reset with the code

```
POST /api/v1/auth/password/reset
```

```
{
  "email": "customer@example.com",
  "code": "4821",
  "new_password": "MyNewPass1!"
}
```

```
{ "success": true, "data": { "message": "Password updated", "sessions_rev
```

This does not sign the user in, and it signs them out everywhere else.

All existing sessions are revoked – if an attacker was in the account, they are now out. Clear any stored tokens and send the user to the login screen.

Situation	Response	<code>error.code</code>
Wrong/expired code	400	INVALID_OTP
Too many attempts	429	RATE_LIMITED

Note the reset uses the **same 4-digit code mechanism** as signup, not a magic link.

5. Session management

Refreshing an expired access token

Access tokens last 30 minutes. When any call returns 401 with

```
error.code == "TOKEN_EXPIRED":
```

```
POST /api/v1/auth/refresh
```

```
{ "refresh_token": "<stored refresh token>" }
```

Returns a **new pair**. Persist both immediately, then retry the original request once.

```
401 TOKEN_EXPIRED
  → POST /auth/refresh
    200 → store new pair, retry original request
    401 → clear tokens, show login screen
```

Implement this with a mutex. If five requests 401 at once and each fires its own refresh, four of them present a token that has already been rotated – which the server treats as theft and revokes every session. One refresh at a time; queue the rest behind it.

Logout

```
POST /api/v1/auth/logout
```

```
Authorization: Bearer <access_token>
```

```
{ "refresh_token": "<stored refresh token>" }
```

Or to sign out every device (e.g. "log out everywhere" in settings):

```
{ "all_devices": true }
```

Idempotent – calling it twice is not an error. Clear local tokens regardless of the response; a network failure should not trap the user in a logged-in state.

6. Profile & security

Who am I?

```
GET /api/v1/users/me
```

```
Authorization: Bearer <access_token>
```

Call this on app launch when restoring a saved token – it confirms the token is still valid and returns the current profile in one round trip.

Change password (signed in)

```
POST /api/v1/users/me/password
```

```
Authorization: Bearer <access_token>
```

```
{ "current_password": "MyFirstPass1!", "new_password": "MyNewPass1!" }
```

```
{ "success": true,  
  "data": { "message": "Password updated", "sessions_revoked": 2,  
            "tokens": { "access_token": "...", "refresh_token": "..." } } }
```

Returns a fresh token pair – you must store it. Every session including your own is revoked, then a new one is issued to the caller. Other devices are signed out; this one stays in. If you ignore `tokens`, your next request fails with `401`.

Situation	Response	<code>error.code</code>
<code>current_password</code> missing when one exists	400	VALIDATION_FAILED
<code>current_password</code> wrong	401	INVALID_CREDENTIALS

Security state

```
GET /api/v1/users/me/security
```

```
{ "success": true,  
  "data": { "is_2fa_enabled": false, "is_biometrics_enabled": true,  
            "biometric_device_count": 1, "has_password": true } }
```

Drives the toggles on your Security screen. `has_password` is useful for OTP-only accounts that never set one.

7. Two-factor authentication

Enabling

1. `POST /auth/2fa/generate` → { `secret`, `qr_code_url` }
2. Show the QR (`qr_code_url` is an `otpauth://` URI) – also show ``secret`` for manual entry
3. User enters the 6-digit code from their authenticator
4. `POST /auth/2fa/enable { code }`

2FA is **not active** until step 4 succeeds. If the user abandons at step 2, nothing changes – deliberately, so a failed enrolment cannot lock them out.

Logging in with 2FA active

```
POST /auth/login → { requires_2fa: true, temp_token, expires_in: 300 }  
↓ user enters their 6-digit code
```

```
POST /auth/login/2fa { temp_token, code } → { tokens, user }
```

`temp_token` is valid for **5 minutes** and works only on `/auth/login/2fa`. It is not an access token — do not send it as a Bearer header, it will be rejected.

Disabling

```
POST /api/v1/auth/2fa/disable
Authorization: Bearer <access_token>

{ "code": "123456" }
```

A current authenticator code is required. Being signed in is not enough — otherwise a stolen phone could simply switch 2FA off.

8. Biometric setup

The crypto contract. Get this exactly right or signatures will not verify.

Key generation

Platform	Algorithm	Notes
iOS	ES256	Secure Enclave supports P-256 ECDSA only
Android	ES256 or ED25519	Keystore, <code>setUserAuthenticationRequired(true)</code>

Generate a key pair that requires biometric authentication to use. The private key never leaves the secure hardware; you only ever send the public key.

Public key format

Base64-encoded DER `SubjectPublicKeyInfo`.

- **Android:** `publicKey.getEncoded()` is already SPKI → base64 it. Done.
- **iOS:** `SecKeyCopyExternalRepresentation` returns a **raw EC point** (`04 || X || Y`), not SPKI. You must wrap it in the SPKI ASN.1 header before sending. This is the single most common integration bug — if the server rejects your key, this is why.

Signing

- Message: the **raw UTF-8 challenge string**, exactly as received. Not a hash of it, not the base64 of it.
- `ES256` : SHA-256 digest, **DER-encoded** signature (iOS `.ecdsaSignatureMessageX962SHA256` produces this).
- `ED25519` : raw 64-byte signature.
- Send it base64-encoded.

device_id

Any stable string unique per app install. It is the lookup key for the enrolled credential, so it must survive app restarts but should change on reinstall.

Re-enrolment

Calling `/auth/biometrics/enable` again for the same `device_id` replaces the key and clears any lockout. That is the recovery path for a device locked after 5 failed attempts: sign in with a password, then re-enrol.

9. Error reference

HTTP	error.code	Meaning	What the app should do
400	VALIDATION_FAILED	Malformed body	Show <code>details[]</code> against the fields
400	INVALID_OTP	Wrong or expired code	Let them retry or resend
401	UNAUTHORIZED	Missing/invalid token	Refresh, then log in
401	TOKEN_EXPIRED	Access token aged out	Refresh and retry once
401	INVALID_CREDENTIALS	Wrong password / signature	Generic "incorrect" message
403	FORBIDDEN	Wrong role, or device locked	Explain, don't retry
404	NOT_FOUND	No such resource	—
409	CONFLICT	Phone taken, 2FA already on	Show a specific message
429	RATE_LIMITED	Too many attempts	Back off using <code>Retry-After</code>

HTTP	error.code	Meaning	What the app should do
500	INTERNAL_ERROR	Server fault	Generic error + the X-Request-ID
501	NOT_IMPLEMENTED	Endpoint not built yet	Feature-flag it off

10. Endpoint summary

Method	Path	Auth	Purpose
POST	/auth/otp/send	—	Send a 4-digit code
POST	/auth/otp/verify	—	Verify → tokens
POST	/auth/login	—	Email + password
POST	/auth/login/2fa	temp	Complete 2FA login
POST	/auth/biometrics/challenge	—	Get a nonce to sign
POST	/auth/biometrics/login	—	Sign in with a biometric
POST	/auth/refresh	—	Rotate tokens
POST	/auth/logout	✓	End session(s)
POST	/auth/password/forgot	—	Request a reset code
POST	/auth/password/reset	—	Reset with the code
POST	/auth/biometrics/enable	✓	Enrol a device key
DELETE	/auth/biometrics/disable	✓	Un-enrol(X-Device-Id header optional)
POST	/auth/2fa/generate	✓	Get TOTP secret + QR
POST	/auth/2fa/enable	✓	Turn 2FA on
POST	/auth/2fa/disable	✓	Turn 2FA off
GET	/users/me	✓	Current user
GET	/users/me/security	✓	2FA / biometric state
PUT	/users/me/profile	✓	Name, phone, avatar
POST	/users/me/password	✓	Set or change password
POST	/auth/register/vendor	—	Register a vendor + restaurant

Method	Path	Auth	Purpose
GET	/admin/restaurants/pending	admin	Approval queue
POST	/admin/restaurants/{id}/verify	admin	Approve or suspend

11. Vendor registration

User story

As a restaurant owner, I want to register my restaurant so I can start taking orders.

1. Enter email → POST /auth/otp/send { "email": ..., "role": "VENDOR" }
2. Code + password + restaurant details
→ POST /auth/register/vendor — tokens —
3. Build the menu (available immediately)
4. Wait for approval (an administrator verifies the restaurant)
5. Open the store → PATCH /vendor/store/status

Note **role: "VENDOR" on step 1** – without it the account is created as a customer, and roles are fixed at creation.

```
POST /api/v1/auth/register/vendor
```

```
{
  "email": "owner@restaurant.com",
  "code": "4821",
  "password": "VendorPass1!",
  "full_name": "Karim Ahmed",
  "restaurant": {
    "name": "Karim's Kitchen",
    "description": "Authentic Bengali cuisine",
    "phone": "+8801712345678",
    "address_line": "House 12, Road 8, Dhanmondi, Dhaka",
    "latitude": 23.7936,
    "longitude": 90.4064,
    "cuisine_types": ["Bengali", "Biryani"]
  }
}
```

201 Created:

```
{
  "success": true,
  "data": {
    "tokens": { "access_token": "...", "refresh_token": "..." },
    "user": { "role": "VENDOR", "email": "owner@restaurant.com" },
    "restaurant": { "id": "...", "slug": "karims-kitchen",
                    "is_verified": false, "status": "CLOSED" },
    "next_step": "Your restaurant is awaiting approval..."
  }
}
```

is_verified: false is expected. The vendor is signed in and can build their menu immediately, but customers cannot see the restaurant until an administrator approves it. Show the **next_step** message and a pending badge.

Situation	Response	error.code
Email already a customer account	409	CONFLICT
Account already has a restaurant	409	CONFLICT
role of ADMIN or RIDER on step 1	400	VALIDATION_FAILED

Rider and admin accounts are **not** self-service – they are created by an administrator.

12. Roles

The system has four roles. **A role is fixed when the account is created** and cannot be changed afterwards – the database enforces this with composite foreign keys, so an account that owns a restaurant cannot quietly become a customer.

Role	How an account is created	Status
CUSTOMER	Self-service – <code>/auth/otp/send</code> (default)	✅ available
VENDOR	Self-service – <code>role: "VENDOR"</code> , then admin approval	✅ available
ADMIN	Server shell only (<code>scripts/create_admin.py</code>)	✅ available
RIDER	Not yet implemented	❌ blocked

Practical consequence for your UI: **one email address is one role**. Someone who signed up as a customer and later wants to run a restaurant must use a different address. Say so plainly at the point of failure – the 409 message already explains it, so surface `error.message` rather than a generic string.

The access token carries the role, and `GET /users/me` returns it. Use it to decide which app shell to render if you ship a single binary for customers and vendors.
